

Ayudantía 5

OOP y Abstracción

Javier Larré

Pontificia Universidad Católica de Chile

javier.larre@uc.cl

24 de abril de 2026

Ayudantía de hoy

- 1 Sobre la I1
- 2 Resumen de la semana
- 3 Ejercicios
- 4 Referencing

Temas que pueden preguntar

- Sintaxis
- Nombres y Bindings
- Tipos de datos
- Estructuras de control sobre secuencias
- Condicionales, loops y recursión
- Subprograms
- OOP
- Jueves 30 de abril
- Hora: 17:30 hrs.
- Duración: 2hrs (+1 PIANE)
- Pueden llevar apuntes escritos a mano, por ambos lados de la hoja

Pregunta esencial

Qué es la abstracción en software?



Qué es la abstracción en software?

- *An abstraction is a view or representation of an entity that includes only the most significant attributes.*
[Sebesta, 2023]
- *Un TDA es una cápsula opaca (clases)*
- Significa que el tipo de dato puede ser usado sin conocer la implementación de sus operaciones.



Qué es la abstracción en software?

- *An abstraction is a view or representation of an entity that includes only the most significant attributes.*
[Sebesta, 2023]
- *Un TDA es una cápsula opaca (clases)*
- Significa que el tipo de dato puede ser usado sin conocer la implementación de sus operaciones.

```
void main() {  
    int topOne;  
    Stack stk; /** Create an instance of  
    stk.push(42);  
    stk.push(17);  
    topOne = stk.top();  
    stk.pop();  
    /** More code  
}
```

Qué es la abstracción en software?

- *An abstraction is a view or representation of an entity that includes only the most significant attributes.*
[Sebesta, 2023]
- *Un TDA es una cápsula opaca (clases)*
- Significa que el tipo de dato puede ser usado sin conocer la implementación de sus operaciones.

```
#include <iostream.h>
```

```
class Stack {  
    private:  
        /** These members are visible only to  
        /** members and friends (see Section  
        int *stackPtr;  
        int maxLen;  
        int topSub;  
    public: /** These members are visible  
    Stack() { /** A constructor  
        stackPtr = new int [100];  
        maxLen = 99;  
        topSub = -1;  
    }
```

```
~Stack() {delete [] stackPtr;}; /** A
```

Qué es la abstracción en software?

- *An abstraction is a view or representation of an entity that includes only the most significant attributes.*
[Sebesta, 2023]
- *Un TDA es una cápsula opaca (clases)*
- Significa que el tipo de dato puede ser usado sin conocer la implementación de sus operaciones.

```
class Stack {  
private:  
    int *stackPtr;  
    int maxLen;  
    int topSub;  
public: /** These members are visible to clients */  
    Stack() { /** A constructor */  
        stackPtr = new int [100];  
        maxLen = 99;  
        topSub = -1;  
    }  
  
    ~Stack() {  
        delete [] stackPtr;  
    }; /** A destructor */  
  
    void push(int number) {  
        if (topSub == maxLen)  
            cerr << "Error in push--stack is full\n";  
        else stackPtr[++topSub] = number;  
    }  
  
    void pop() {  
        if (empty())  
            cerr << "Error in pop--stack is empty\n";  
        else topSub--;  
    }  
  
    int top() {  
        if (empty())  
            cerr << "Error in top--stack is empty\n";  
        else  
            return (stackPtr[topSub]);  
    }  
  
    int empty() {  
        return (topSub == -1);  
    }  
}
```


Pregunta de prueba

Supongamos que ahora la operación `top()` retorna el puntero al primer elemento del stack en vez de su valor
¿Esto rompería la abstracción?

```
int* top() {  
    if (empty())  
        cerr << "Error_in_top--stack_is_empty";  
    else  
        return (&stackPtr[topSub]);  
}
```

Pregunta de prueba

Supongamos que ahora la operación `top()` retorna el puntero al primer elemento del stack en vez de su valor

¿Esto rompería la abstracción?

Si, porque permitiría modificar el primer elemento del stack sin utilizar operaciones definidas en la TDA (clase stack, en este caso)

```
int* top() {  
    if (empty())  
        cerr << "Error_in_top--stack_is_empty";  
    else  
        return (&stackPtr[topSub]);  
}
```

¿Cuáles son los **Peligros** si un lenguaje no permite ocultar información?

```
public class Point {  
    public double x;  
    public double y;  
}
```

¿Cuáles son los **Peligros** si un lenguaje no permite ocultar información?

Exponer el interior de nuestros objetos y sus operaciones causa dependencias sobre las implementaciones de este. Estas dependencias no nos permiten hacer **cambios** en los tipos o implementaciones tan fácilmente, cambios que pueden ocurrir impulsivamente y que pudieron haber sido fáciles de hacer. [Martin, 2008]

```
public interface Point {  
    double getX();  
    double getY();  
    void setCartesian(double x, double y);  
    double getR();  
    double getTheta();  
    void setPolar(double r, double theta);  
}
```

Pregunta esencial

¿Cuál es la diferencia entre cada `Point`? ¿Cómo están (o pueden estar) implementados? **¿Cómo puedo modificar los datos en cada caso?**

```
public class Point {  
    public double x;  
    public double y;  
}
```

```
public interface Point {  
    double getX();  
    double getY();  
    void setCartesian(double x, double y);  
    double getR();  
    double getTheta();  
    void setPolar(double r, double theta);  
}
```

Pregunta esencial

¿Cuál es la motivación principal para usar (o implementar) herencia?



Pregunta esencial

¿Cuál es la motivación principal para usar (o implementar) herencia?

Permite reusar código de TDA, ¿pero por qué no solamente las referenciamos y usamos en vez de heredar de ellas?



¿Cuál es la motivación principal para usar (o implementar) herencia?

Permite reusar código de TDA, ¿pero por qué no solamente las referenciamos y usamos en vez de heredar de ellas?

Generalmente, los problemas que el software busca solucionar son casos particulares de problemas más generales. Es decir, especificamos (heredamos) la implementación. Además, también está permitido usar clases heredadas en programas que reciban instancias de las clases padre.

Pregunta esencial

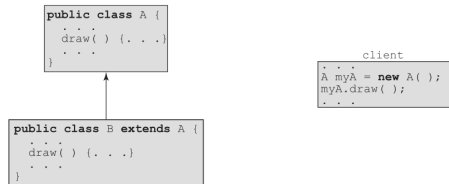
Cuál es la diferencia entre un subprogram cualquiera y un método?



Pregunta esencial

Cuál es la diferencia entre un subprogram cualquiera y un método?

Los métodos permiten *override*



Mucha suerte en la prueba!



References



[Robert Sebesta \(2023\)](#)

Concepts of programming languages



[Robert C. Martin \(2008\)](#)

Clean Code: A Handbook of Agile Software Craftsmanship

The End

Questions? Comments?